

Freebyz Mobile Advertising & Telemetry

Client API Integration & Monetization Guide for Mobile Developers

Version: 1.0 (Production) Platform: Android (Kotlin/Flutter/RN) & iOS (Swift/Flutter/RN) Status: Approved & Live

1. Overview & Architecture

This guide explains how mobile developers can integrate and monetize Freebyz mobile applications (Android & iOS) using the centralized Advertising Backend API. The API gives mobile clients dynamic control over:

- **Kill Switches:** Instantly disable all ads remotely during app reviews or emergencies.
- **Dynamic Placements:** Configure in-feed ad frequency (e.g. insert an ad card after every 5th task).
- **Session Capping:** Protect user experience by enforcing maximum ad impressions per user session.
- **Telemetry & Analytics:** Report ad impressions, clicks, and load failures back to the admin dashboard.

Strict Screen Restrictions: Under no circumstances should ads be rendered on: **(1)** Auth screens (Login, Register, OTP Verification), **(2)** Financial/Wallet screens (Withdrawal, Deposit, Checkout), **(3)** Task submission & proof upload forms.

2. Base URL & Authentication

Environment	Base URL	Access Level
Production API	<code>https://api.freebyz.com</code>	Public Endpoints (No Bearer Token required)
Local / Staging API	<code>http://127.0.0.1:8081</code>	Public Endpoints

3. Endpoint 1: Fetch Advertising Configuration

GET `/api/public/advertising/config`

Call this endpoint on app launch or when entering major listing screens. It returns global kill switches, active placement rules, and ad code/links.

Query Parameters

Parameter	Type	Required	Description & Accepted Values
<code>platform</code>	string	Yes	<code>android_app</code> or <code>ios_app</code>
<code>device</code>	string	No	<code>mobile</code> (default: <code>all</code>)
<code>page_type</code>	string	No	Filter by page: <code>TASKS_LIST</code> , <code>JOBS_LIST</code> , etc. (Omit to get all).

Sample Response Body (`200 OK`)

```
{
  "status": true,
  "message": "Advertising configuration retrieved successfully.",
  "data": {
    "enabled": true,
    "provider": "adsterra",
    "platform": "android_app",
    "max_ads_per_session": 10,
    "formats": {
      "native": true,
      "banner": true,
      "social_bar": false,
      "interstitial": false,
      "popunder": false,
      "smartlink": false
    },
    "placements": [
      {
        "key": "TASKS_LIST_MID_NATIVE",
        "page_type": "TASKS_LIST",
        "format": "native_banner",
        "device": "all",
        "platform": "all",
        "position": "AFTER_TASK_5",
        "display_interval": 5,
        "frequency_cap": 3,
        "priority": 1,
        "enabled": true,
        "code": "https:// ..."
      }
    ]
  }
}
```

4. Endpoint 2: Telemetry & Analytics Event Logging

POST /api/public/advertising/events

Send lightweight beacons whenever an ad is displayed (impression), tapped (click), or fails to load. This populates the Admin portal with real-time performance analytics.

Request Payload Fields

Field	Type	Required	Description
event_name	string	Yes	Must be one of: <code>ad_rendered</code> , <code>ad_clicked</code> , <code>ad_failed</code> , <code>ad_requested</code>
placement_key	string	Yes	The key from config (e.g. <code>TASKS_LIST_MID_NATIVE</code>)
page_type	string	Yes	Screen identifier (e.g. <code>TASKS_LIST</code> , <code>JOBS_LIST</code>)
ad_format	string	No	<code>NATIVE_BANNER</code> or <code>STANDARD_BANNER</code>
platform	string	No	<code>android_app</code> or <code>ios_app</code>
device_type	string	No	<code>mobile</code>

Example JSON Payloads

1. Ad Rendered (Impression)

```
{
  "event_name": "ad_rendered",
  "placement_key": "TASKS_LIST_MID_NATIVE",
  "page_type": "TASKS_LIST",
  "ad_format": "NATIVE_BANNER",
  "platform": "android_app",
  "device_type": "mobile"
}
```

2. User Clicked Ad

```
{
  "event_name": "ad_clicked",
  "placement_key": "TASKS_LIST_MID_NATIVE",
  "page_type": "TASKS_LIST",
  "ad_format": "NATIVE_BANNER",
  "platform": "android_app",
  "device_type": "mobile"
}
```

5. Mobile Client Implementation Flow

Step 1: Session Management & Kill Switch Check

1. On application startup, fetch and cache the configuration JSON.
2. Check `data.enabled`. If `false`, disable all advertising components.
3. Initialize a session ad counter: `sessionAdCount = 0`.
4. Before rendering any ad, verify: `sessionAdCount < data.max_ads_per_session`.

Step 2: In-Feed Ad Insertion (Available Tasks / Jobs List)

For list feeds (e.g., RecyclerView or ListView of Available Tasks), check `placement.display_interval` (e.g. 5):

- Render standard task card at index 0, 1, 2, 3, 4.
- At index 5 (after 5 tasks), insert the **Sponsored Ad Card** item.
- Increment `sessionAdCount++` and immediately fire `POST /api/public/advertising/events` with `ad_rendered`.
- When user taps the ad card, fire `POST /api/public/advertising/events` with `ad_clicked` and open the link in external browser.

6. Code Implementation Examples

Flutter (Dart) Example Implementation

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class FreebyzAdService {
  static const String baseUrl = 'https://api.freebyz.com';
  static Map? _config;
  static int _sessionImpressions = 0;

  // 1. Fetch Config on App Start
  static Future initConfig() async {
    try {
      final response = await http.get(
        Uri.parse('$baseUrl/api/public/advertising/config?platform=android_app'),
      );
      if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        _config = data['data'];
      }
    } catch (e) {
      print('Failed to load ad config: $e');
    }
  }

  // 2. Check if Ad Can Be Displayed
  static bool canShowAd() {
    if (_config == null || _config!['enabled'] != true) return false;
    final int maxAds = _config!['max_ads_per_session'] ?? 10;
    return _sessionImpressions < maxAds;
  }

  // 3. Log Telemetry Event
  static Future logEvent({
    required String eventName,
    required String placementKey,
    required String pageType,
  }) async {
    try {
      await http.post(
        Uri.parse('$baseUrl/api/public/advertising/events'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
          'event_name': eventName,
          'placement_key': placementKey,
          'page_type': pageType,
          'platform': 'android_app',
          'device_type': 'mobile',
        }),
      );
      if (eventName == 'ad_rendered') _sessionImpressions++;
    } catch (_) {}
  }
}
```

7. Mobile Developer QA & Verification Checklist

Check	Requirement	Expected Result
1	Kill switch test	When admin sets status to INACTIVE, no ads display in mobile app.
2	Session capping test	Ads stop rendering after reaching <code>max_ads_per_session</code> (e.g. 10).
3	In-feed spacing	Ads appear precisely after every 5th task without layout overflow.
4	Telemetry tracking	Admin panel increments impressions and clicks accurately in real-time.
5	Excluded screens	Zero ads on Login, Registration, OTP, and Wallet withdrawal pages.

Backend Support: If you need custom placement keys or additional parameters for new app features, contact the backend team or adjust placement settings via the Admin Portal at `/admin/advertising` .